Portfolio Asset Identification using Graph Algorithms on a Quantum Annealer

Shravar G Tanawde(M25IQT012)

November 12, 2025

1 Background Of The Paper

One of the core pillars of modern finance is Portfolio optimization, which requires identifying correlated and uncorrelated assets within a universe of assets. This type of correlation clustering enables the portfolio manager to select assets to implement different diversification and hedging strategies. The current classical approach to this problem, such as Markowitz's Mean-Variance Optimization (MVO), is computationally expensive for large asset sets due to their combinatorial nature. As the number of possible asset combinations grows exponentially, finding the actual optimal portfolio quickly becomes infeasible for classical algorithms.

Therefore, to address the scalability issue of the classical approach, a quantum approach has emerged as a popular solution to NP-hard problems. D-Wave's quantum annealer enables the encoding of such problems into an Ising Hamiltonian or Quadratic Unconstrained Binary Optimization (QUBO) form, which the annealer can solve through quantum processes.

This paper, "Portfolio Asset Identification using Graph Algorithms on a Quantum Annealer" by Angad Kalra, Faisal Qureshi, and Michael Tisi, explores encoding graph-based algorithms into Ising or QUBO formulations, which the authors implements on a quantum annealer to identify optimal or near-optimal portfolios.

2 Summary Of The Paper

The paper presents a method for Portfolio Asset Identification using various Graph Algorithms by representing financial assets as nodes in a correlation graph, and edges represent strong correlation between assets. The goal is to identify a set of nodes (assets) that are weakly correlated, which enhances diversification and hence allows the portfolio manager to make informed decisions while hedging the money.

The authors focus on four such Graph Algorithms, namely the Maximum Clique Problem (MCP), the Maximum Independent Set Problem (MISP), the Minimum Graph Coloring Problem (MGCP), and the Structural Balance Problem (SBP). Afterward, MCP/MISP and MGCP are encoded into the QUBO formulation, and SBP into the Ising formulation, which are then solved using D-Wave's quantum annealing-based 2000 qubit machine, the D-Wave 2000Q. The primary focus of this research is to elucidate the capabilities of quantum algorithms to perform computationally practical tasks and then to assess the quality of quantum solutions in comparison to that of classical algorithm solutions. This does not compare the difference in computational speed between the methods.

The results show that quantum annealing can achieve comparable or better diversification outcomes compared to classical heuristics, with significant advantages in scalability and computational time for complex, large asset universes. As the density of data increases, the results have been of consistently superior quality compared to their classical counterpart.

3 Why This Topic Is Interesting

- It bridges quantum computing and fintech, two high-impact fields rarely combined effectively at the time.
- Demonstrates industrial use of quantum hardware such as the D-Wave quantum annealer.
- Addresses a real-world NP-hard problem (portfolio optimization) that is central to financial technology.
- Illustrates how graph theory and quantum optimization intersect, creating a framework that can generalize other problems, such as risk clustering or network-based asset selection.

4 Research Gap Addressed

According to the authors of this paper, the study presented is the first to apply graph algorithms to D-Wave in the domain of financial portfolio management. Prior to this paper, most research in quantum finance focused on theoretical formulations of portfolio optimization using simulated annealing or quantum-inspired algorithms, rather than running on actual quantum hardware. This paper also conducts a comparative study of four different algorithms and their solutions, utilizing both classical and quantum approaches.

Additionally, any real-life portfolio optimization prior to this paper focused primarily on using classical algorithms, such as Mean-Variance optimization, Hierarchical clustering, and correlation-based methods. These methods faced key limitations:

1. Computation Complexity
2. Local minima issues
3. Scalability bottleneck

Hence, this paper fills several gaps:

- Demonstrates a real-world implementation of asset selection on a quantum annealer.
- Introduces a graph-based representation of financial correlations suitable for quantum encoding.
- Provides empirical evidence comparing quantum annealing performance to classical clustering methods.
- Suggests a scalable alternative to traditional optimization under large-dimensional financial data.

5 Alternative Approach

This paper was one of the first to explore a quantum solution for portfolio optimization. The only alternative at that time was classical algorithms such as:

1. Mean–Variance Optimization (Markowitz Model): Foundational model balancing expected return and variance, but computationally intensive for large asset sets.
2. Heuristic Algorithms: Genetic algorithms, simulated annealing, and particle swarm optimization are used to approximate solutions but are prone to local minima.
3. Hierarchical Risk Parity (HRP): Introduced by López de Prado (2016), uses hierarchical clustering on correlation matrices for diversification.
4. Spectral Clustering & Graph Methods: Used to detect asset communities in correlation networks.

Along with this, there were a few quantum approaches to portfolio optimization as well, but none of them used a graph-based algorithms to map out the problem. These approaches were:

1. Quantum Annealing for Portfolio Optimization (Rosenberg et al., 2016–2017): D-Wave's own team demonstrated early formulations of the portfolio optimization problem as a QUBO, focusing on minimizing variance and maximizing expected return. This paper was built on top of this approach.

2. Quantum Boltzmann Machines (QBM): Uses quantum-inspired machine learning methods for financial modeling.

3. Variational Quantum Eigensolver (VQE) for Optimization: VQE-based hybrid approaches began to be proposed for general optimization tasks, including risk-return optimization, marking the start of gate-based portfolio algorithms.

However, after this paper, more quantum based approach began to emerge to tackle this problem. A few of these alternative methods were:

1. Quantum Approximate Optimization Algorithm (QAOA): The portfolio problem was cast as a constrained binary optimization mapped to a QAOA circuit. This method improved flexibility and applicability on noisy devices.

2. Hybrid Quantum–Classical Optimizers: Uses classical pre-processing(covariance matrix reduction) followed by quantum optimization for fine-tuning asset selection.

3. Quantum Reinforcement Learning (QRL)

4. Quantum Graph Neural Networks (QGNNs)

6 Methodology Used In The Paper

6.1 Problem formulation and Mapping to Graph Structure

The authors aim to identify sets of assets whose returns are weakly correlated, thereby achieving better diversification. Assets represents nodes in a graph. The edges reflect correlation metrics between assets. So for creating graph and identifying it is nodes and edges, first we assume dataset D consists of a set of K assets $A = \{a_k\}$ for $k \in \{1, ..K\}$, and corresponding daily prices of these assets, $price(a_k^{(t)})$ for $t \in daterange[T]$ for some time period T. To model how prices change over time, we normalize $price(a_k^{(t)})$ into a daily price return $x_k^{(t)}$ as follows:

$$DailyPriceReturn = x_k^{(t)} = \frac{price(a_k^{(t)}) - price(a_k^{(t-1)})}{price(a_k^{(t-1)})} \tag{1}$$

$$SampleCovarianceMatrix = S_m = 1/N \sum_t (x^{(t-i)} - \bar{x})(x^{(t-i)} - \bar{x})^T \tag{2}$$

$$where\ N = NumberOfDaysInMonth \tag{3}$$

$$SampleCorrelationMatrix = C_m = (diag(S_m))^{1/2}(diag(S_m))^{1/2} \tag{4}$$

Here the value of C_m determines the correlation between 2 assets. If this correlation is above a certain threshold the an edge is formed. This allows us to check the density of edges at any given time by,

$$Density(G) = \frac{2|\varepsilon|}{|\nu|(|\nu| - 1)} \tag{5}$$

6.2 Graph Algorithms

Once the nodes and edges of the graph are established, the author compares four graph algorithms' solutions using classical and quantum approaches. These algorithms are:

1. **Maximum Clique and Maximum independent Set**: The maximum clique problem (MCP) is to find the clique with maximum cardinality in a given graph. The maximum independent set problem (MISP) involves finding the largest independent set within a graph. Both of these problems are equivalent to each other as the solution to one problem is equal to the complement of the solution to other. Classically, MCP and MISP employ some variation of branch-and-bound method and heuristic approaches that include employ greedy search, localized search or genetic methods.
2. **Minimum Graph coloring**: The Graph coloring Problem (GCP) is to label (color) the vertices of graph such that no two vertices connected by an edge have the same label (color). The Minimum Graph coloring Problem (MGCP) is to find for a given graph, the graph color with the least number of labels (colors). Major classical approaches to solving MGCP include greedy coloring, subgraph expansion and recursive methods.
3. **Structural Balance**: . The Structural Balance Problem (SBP) is to calculate the frustration index of a signed graph. A balanced graph is graph that can be divided into 2 cluster in a way that all vertices in a cluster are connected to each other by positive edges and any connection between vertices of different cluster is a negative edge. Author refers to other research paper for classical solution to this problem which includes Binary Linear Programming for frustration index, correlation-clustering heuristics, etc.

6.3 Encoding into Annealer

After establishing these graph problems, they were encoded into QUBO formulations in case of MISP/MCP and MGCP, and SBP was encoded into an Ising formulation. D-Wave machines use quantum annealing to find the lowest energy eigenvalue of the Ising Hamiltonian, H_{Ising}:

$$\mathcal{H}_{\text{Ising}} = -\frac{A(s)}{2}\left(\sum_i \sigma_x^{(i)}\right) + \frac{B(s)}{2}\left(\sum_i h_i\sigma_z^{(i)} + \sum_{i>j} J_{i,j}\sigma_z^{(i)}\sigma_z^{(j)}\right) \tag{6}$$

Therefore, D-Wave requires a problem to be mapped into Ising or QUBO objective function, which are defined respectively, as follows:

$$E_{\text{Ising}}(\vec{s}) = \sum_{i=1}^{N} h_i s_i + \sum_{i=1}^{N}\sum_{j=i+1}^{N} J_{i,j}s_i s_j \tag{7}$$

$$f(x) = \sum_i Q_{i,i}x_i + \sum_{i<j} Q_{i,j}x_i x_j = \min_{x\in\{0,1\}^n} x^T Q x \tag{8}$$

For purposes of mapping, The coefficient matrices of MISP, SBP and MGCP are given below:

$$Q_{i,j}^{MISP} = \begin{cases} 1, & \text{if } (i,j)\in\mathcal{E}, \text{ and } i<j \\ 0, & \text{otherwise} \end{cases} \tag{9}$$

$$J_{i,j}^{SBP} = \begin{cases} \text{sign}(i,j), & \text{for } (i,j)\in\mathcal{E}, \text{ and } i<j \\ 0, & \text{otherwise} \end{cases} \tag{10}$$

$$Q_{i,j}^{MGCP} = \begin{cases} 2A, & \text{if } i,j \text{ correspond to the same vertex but different colors,} \\ B, & \text{if } i,j \text{ correspond to adjacent vertices assigned the same color,} \\ 0, & \text{otherwise.} \end{cases} \tag{11}$$

$$\textit{where } A >> B \text{ and values of A and B are dependent on penalty} \tag{12}$$

With these coefficients, substituted into QUBO or Ising formulation as required, they are solved using D-Wave's quantum annealer. Results obtained in this way are compared to results that are known from classical methods, to study the difference in quality of solutions.

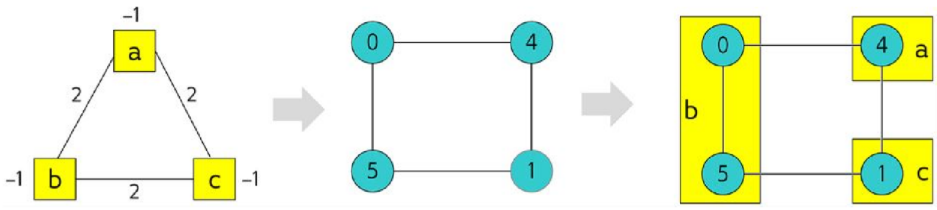


Figure 1: QUBO Mapping and Minor Embedding

The left most figure maps QUBO function $E(x, y, z) = 2xy + 2xz + 2yz - x - y - z + 1$ on to a graph. The next two figures "minor embed" the QUBO into D-Wave's Chimera Architecture.

7 Setup And Procedure

- **Computing Resource** Author uses Ocean Software, a python based library offered by D-Wave to interact with the D-Wave 2000Q machine, which uses D-Wave Chimera Architecture. Comparatively, classical algorithms were run on a 2015 Macbook Pro with Intel i5 2.7 GHz CPU and 8 GB RAM.

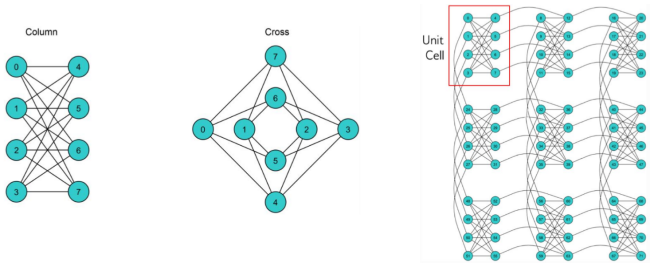


Figure 2: Enter D-Wave Chimera Architechture

- **Graph Algorithm** Quantum annealing based algorithm provided in the Ocean Software library without any special performance tuning. The authors implemented a Graph data structure in the Python NetworkX graph library.
- **Data** Four financial datasets, namely Asset Class, Sectors, SP, and FTSE, were prepared. They were sourced from the Center for Research in Security Prices. For each dataset, 2519 days (10 years) of daily security prices were retrieved.

Quantum (dwave_networkx)	**Classical (networkx.algorithms)**
maximum clique()	approximation.clique.max clique()
maximum independent set()	approximation.clique.max independent set()
min vertex coloring()	coloring.greedy color()
structural imbalance()	n/a

Table 1: Graph Algorithm

Datasets	**#**	**Description**
Asset Classes	19	major ETFs representing asset classes i.e. equity, debt, RE, commodities etc.
Sectors	11	major ETFs representing sectors i.e. utilities, finance, energy etc.
FTSE	22	stocks from London based FTSE 100 Index
S&P	90	stocks from US based S&P 100 Index

Table 2: Datasets

Dataset	Thresh. Levels	# Months	Total Grpahs
Asset Classes	.5,.6,.7,.8,.9	120	600
Sectors	.5,.6,.7,.8,.9	120	600
FTSE	.5,.6,.7,.8,.9	120	600
S&P	.5,.6,.7,.8,.9	120	600
			2400

Table 3: Thresholded Graphs

8 Results

The objective of the study is to apply quantum algorithms to the real world problem of identifying assets for various financial portfolio strategies, and also to compare the quality of classical and quantum solutions. Results of all four graph algorithms and the result comparison are as follows:

8.1 Maximum Clique Problem

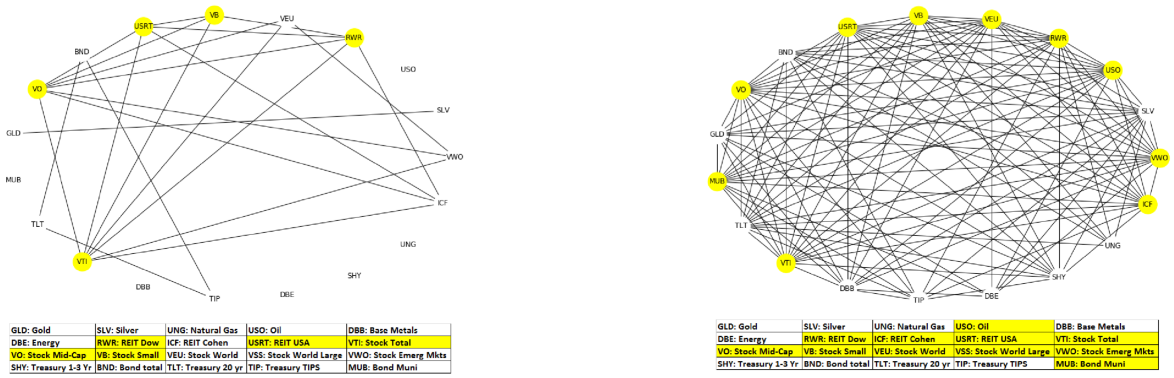


Figure 3: Asset Class, Low Density and High Density

To get a good solution score with MCP, we are looking for the cardinality of the returned maximum clique. Hence, in the case of MCP, a bigger score is better, as is apparent in the graphs shown below. As density increases, MCP scores increase as expected, but it is also apparent that quantum shows better results than classical on all levels of density.

8.2 Maximum Independent Set Problem

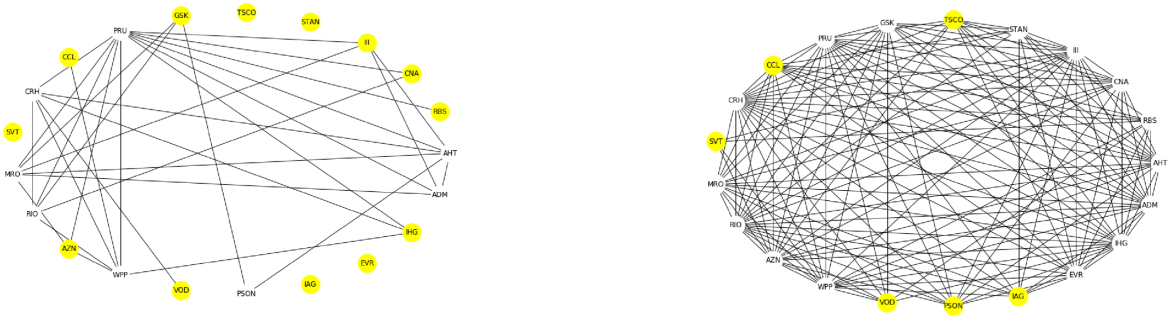


Figure 4: FTSE, Low Density and High Density

Similar to MCP results, the MISP score should be bigger for it to be better. The graphs below show that MISP performs better with the quantum approach, especially at higher densities. Also predictably, MISP scores decrease because independent sets generally become smaller as a graph becomes denser.

8.3 Minimum Graph coloring Problem

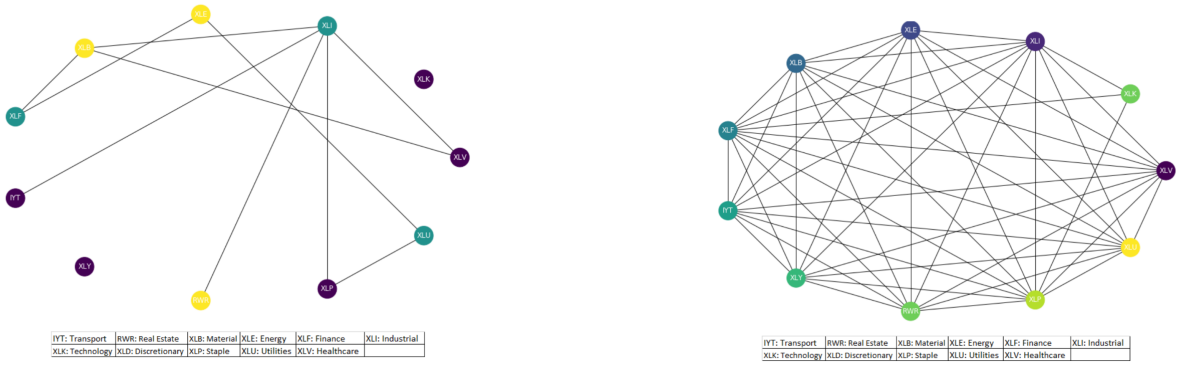


Figure 5: Sectors, Low Density and High Density

Unlike MCP and MISP, MGCP requires a smaller score to be better than its classical counterpart. In this case, both classical and quantum have performed competitively at all densities. As seen in the graphs below, the quantum approach does have a very slight benefit over the classical approach, but the difference is insignificant.

8.4 Structural Balance Problem

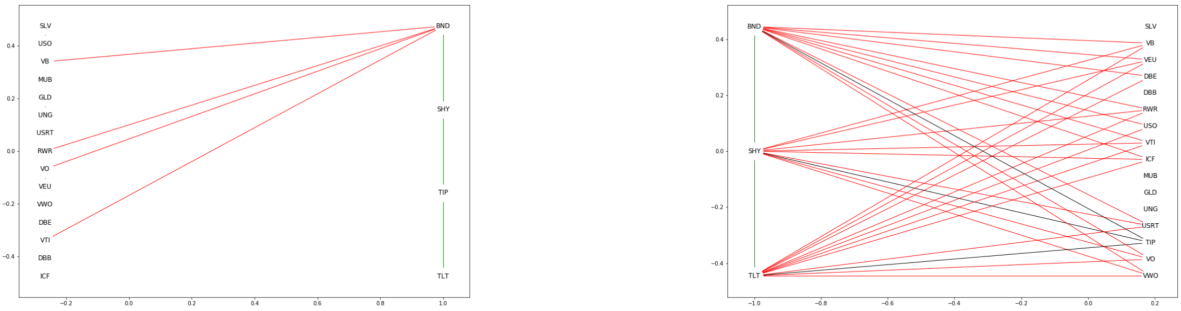
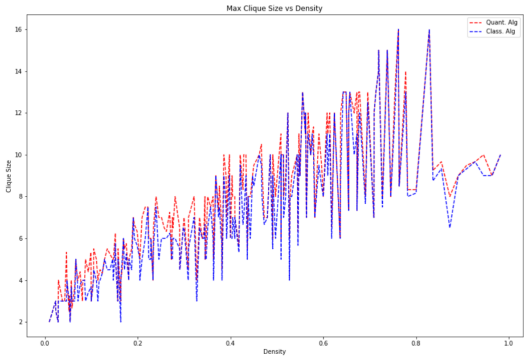
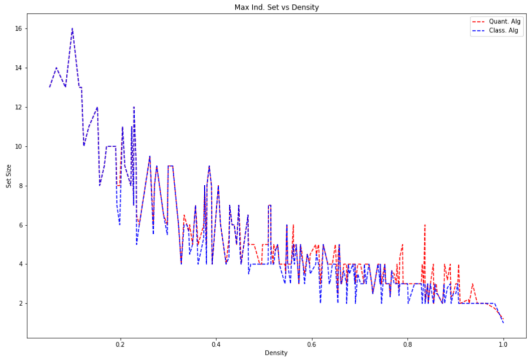


Figure 6: Asset Class, Low Density and High Density

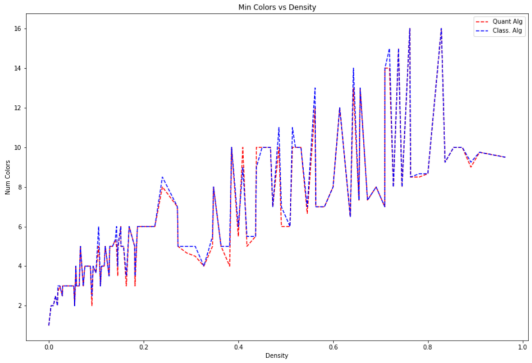
Unlike the previous three cases, SBP was not compared to the classical approach, and only quantum data is available. Similar to MGCP, SBP also requires a smaller score to be better. An interesting result is observed in the graph of SBP, that is, there is low frustration at low density due to the ease of balancing such a network. However, the frustration reaches its maximum at a density of 0.8, after which it even begins to fall back down. Secondly, there is no discernible pattern to the SBP score, as perfect scores are interspersed among poor scores.



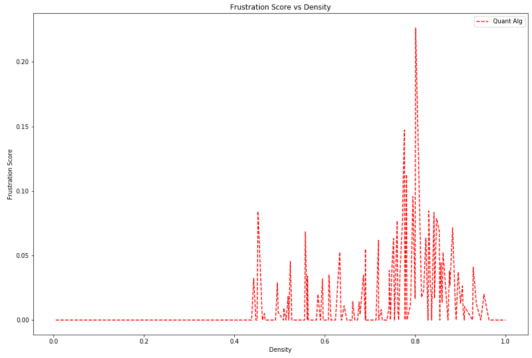
(a) Maximum Clique Number vs. Density



(b) Maximum Independent Set Number vs Density



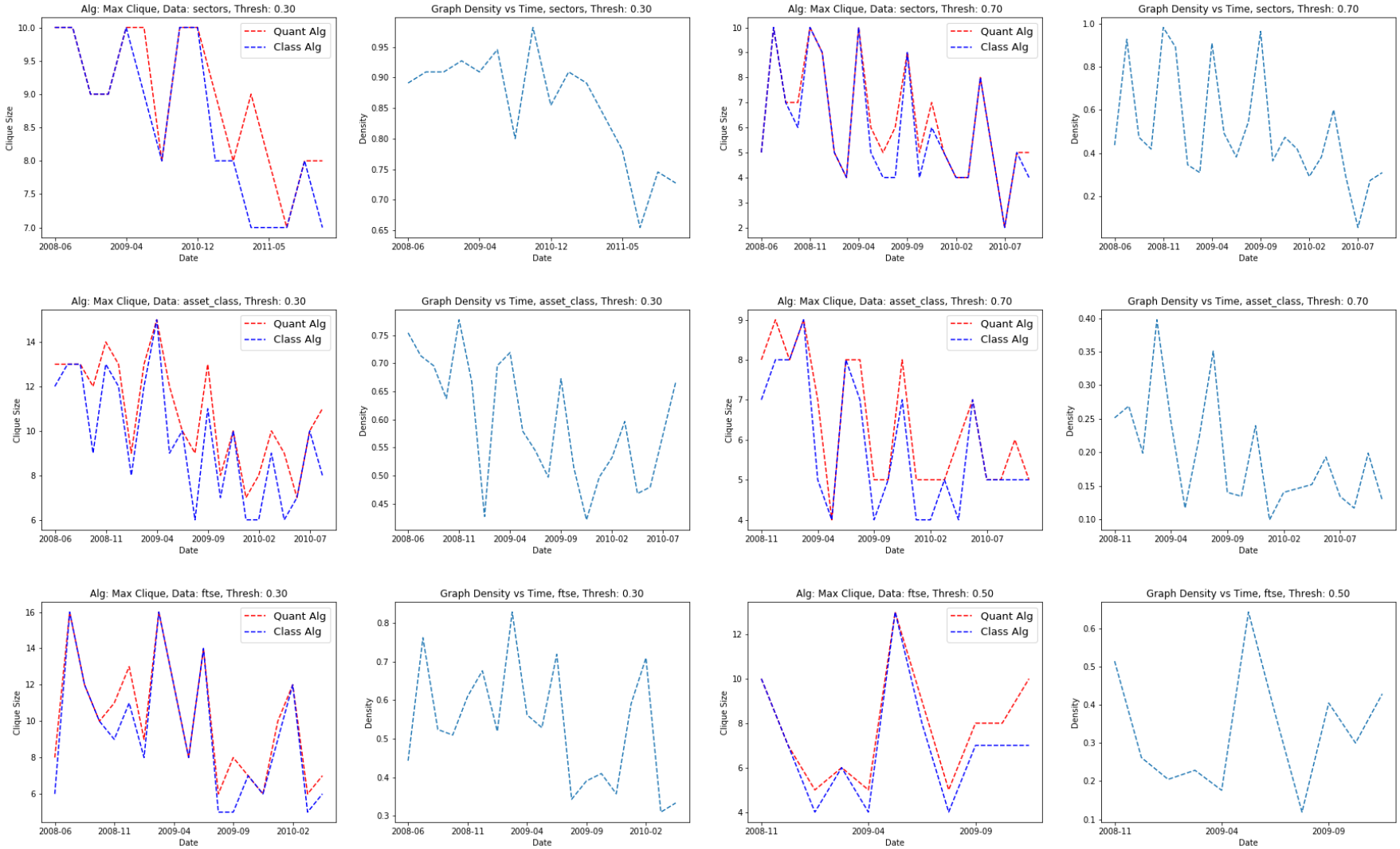
(c) Minimum Graph colors vs. Density



(d) Minimum Frustration Score vs. Density

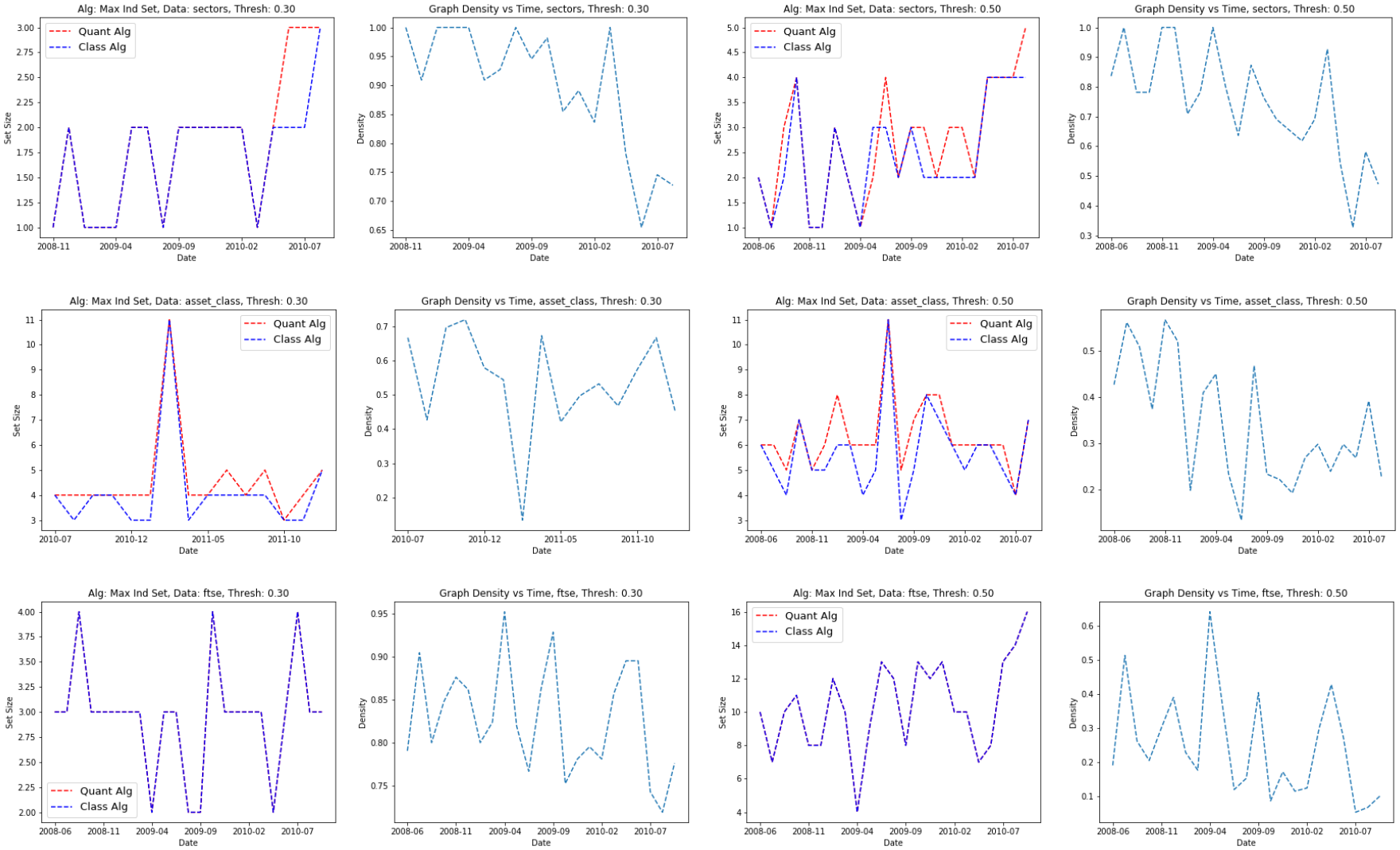
Figure 7: Graph Algorithm Comparison: Score vs. Density

9 Appendix C: Max Clique - Quantum vs. Classical Algorithm Scores



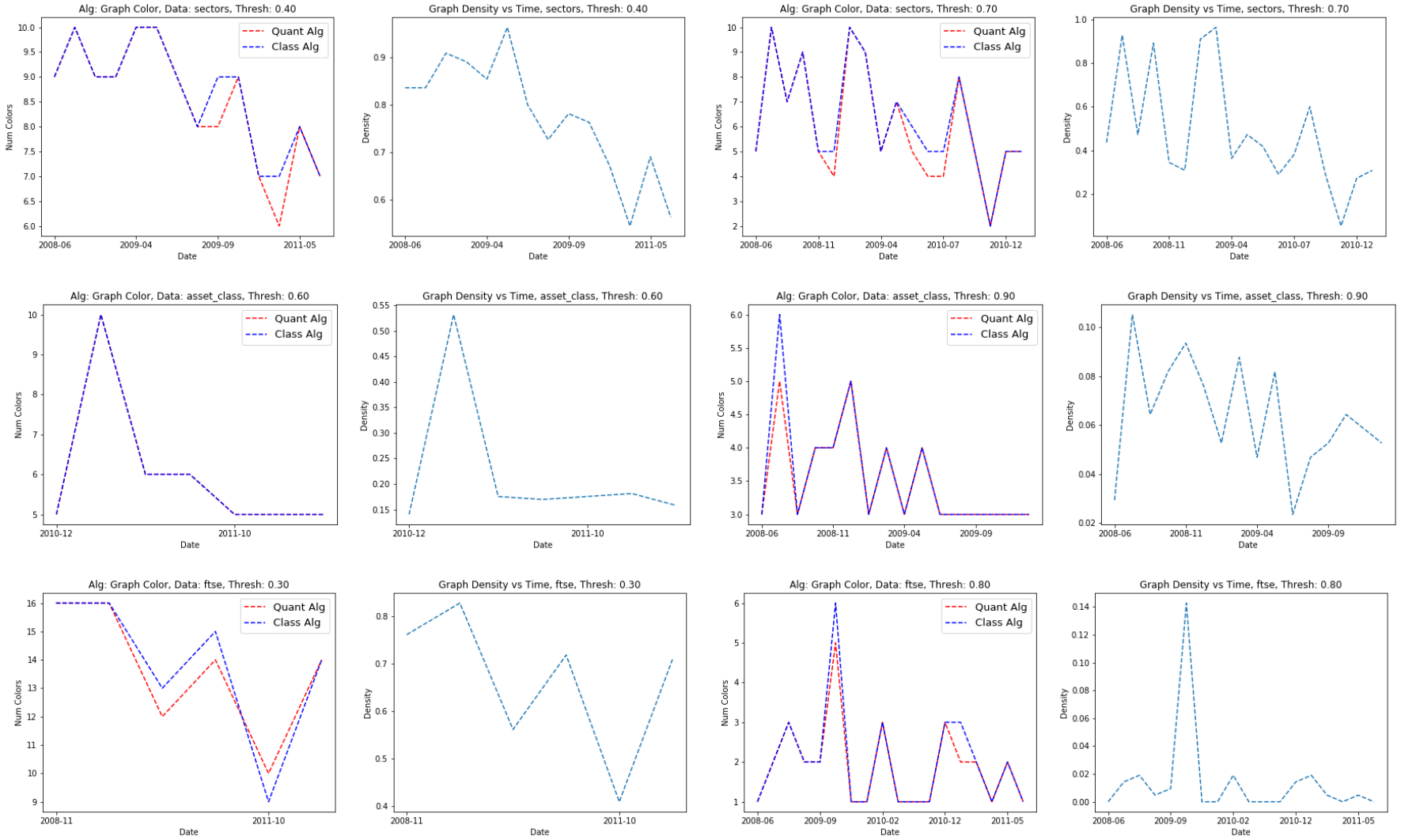
Sizes of max cliques found are plotted alongside density of the underlying graph. For each dataset, high and low density thresholds are displayed.

10 Appendix D: Max Independent Set - Quantum vs. Classical Algorithm Scores



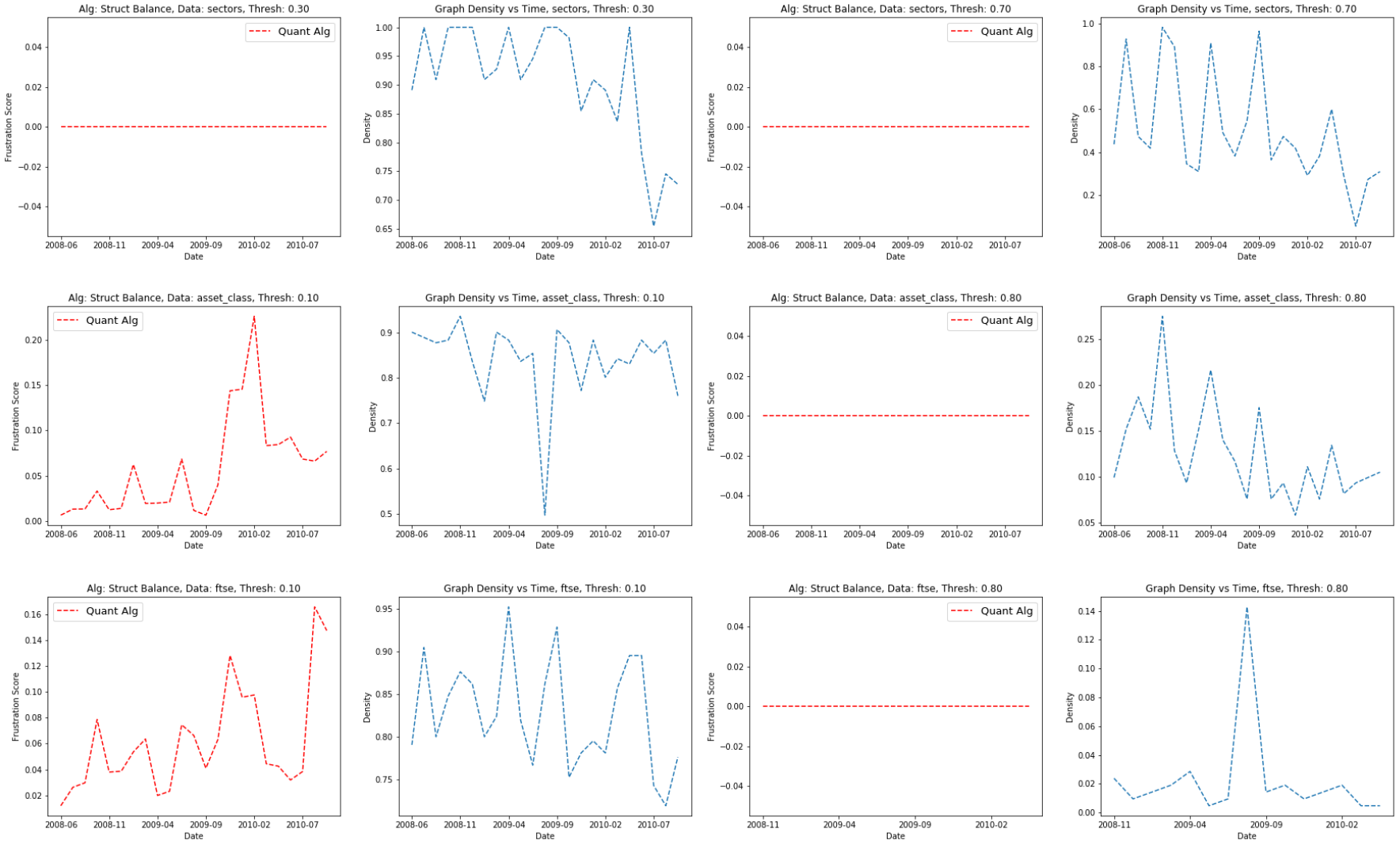
Max indep. set sizes found are plotted alongside density of the underlying graph. For each dataset, high and low density thresholds are displayed.

11 Appendix E: Graph Coloring - Quantum vs. Classical Algorithm Scores



Minimum colourings found are plotted alongside the density of the underlying graph. For each dataset, high and low density thresholds are displayed.

12 Appendix F: Structural Balance - Quantum vs. Classical Algorithm Scores



Frustration Score is plotted alongside the density of the underlying graph. For each dataset, high and low density thresholds are displayed.

9 Reproduction Work and Creative Approach

9.1 Objective

The original work by the authors utilized D-Wave's quantum annealer to solve the QUBO and Ising problems. The problem with reproducing that approach is the lack of connection to the D-Wave hardware. Therefore, I approached this problem using the Quantum Approximate Optimization Algorithm (QAOA) to attempt to solve the QUBO/Ising problem. The objective of this reproduction is to use QAOA to solve the QUBO model, with accuracy and scalability as the final goals.

For the initial attempt at reproduction, I am focusing on the Maximum Clique Problem only, as my goal is to reproduce the quantum annealing solution using QAOA. Depending on the results of this, I will expand it to the other three graph algorithms as well.

9.2 Tools and Framework

- **Libraries Used**: PennyLane, NumPy, NetworkX, yfinance, matplotlib and Python-Louvain
- **Dataset**: Asset data is obtained using yfinance. They are specifically tech-related to increase correlation for testing diversification. The assets chosen for this are [APL, MSFT, NVDA, JPM, MA, XOM, JNJ, PG, KO, CAT, META, DUK]. Only 12 assets were chosen to keep computational resource manageable. As part of further improvement, the asset list was increased to 25 assets, i.e. [AAPL, MSFT, GOOGL, AMZN, NVDA, META, TSLA, JPM, V, MA, XOM, CVX, UNH, HD, PG, KO, PEP, BAC, PFE, DIS, NFLX, INTC, ADBE, NKE, T]

9.3 Methodology

9.3.1 Graph Creation

The first step of reproduction is to get the assets and create a correlation graph between them. For this purpose, I used the *yfinance* library to download real-time data of tech assets, namely, [APL, MSFT, NVDA, JPM, MA, XOM, JNJ, PG, KO, CAT, META, DUK]. Using this data, the Daily Profit Returns and Covariance matrix between the data were calculated.

Covariance data is used to create a Correlation Matrix to determine the relation between assets, which will become the Edge of the graph with assets at its nodes. To reduce the density of edges on the graph, I applied a threshold of 0.45, below which the relation between assets is considered too weak to warrant an edge connection. This gave a graph with a sufficient number of correlation and non-correlation sets.

Table 4: Correlation Matrix of Selected Assets

Ticker	AAPL	CAT	DUK	JNJ	JPM	KO	MA	META	MSFT	NVDA	PG	XOM
AAPL	1.000	0.403	0.269	0.321	0.434	0.356	0.595	0.535	0.705	0.576	0.365	0.311
CAT	0.403	1.000	0.242	0.305	0.616	0.333	0.497	0.286	0.407	0.387	0.233	0.560
DUK	0.269	0.242	1.000	0.505	0.370	0.623	0.368	0.117	0.270	0.086	0.571	0.317
JNJ	0.321	0.305	0.505	1.000	0.364	0.528	0.389	0.158	0.328	0.121	0.545	0.296
JPM	0.434	0.616	0.370	0.364	1.000	0.451	0.590	0.331	0.449	0.362	0.322	0.553
KO	0.356	0.333	0.623	0.528	0.451	1.000	0.501	0.176	0.363	0.157	0.630	0.377
MA	0.595	0.497	0.368	0.389	0.590	0.501	1.000	0.474	0.636	0.490	0.404	0.448
META	0.535	0.286	0.117	0.158	0.331	0.176	0.474	1.000	0.611	0.528	0.192	0.186
MSFT	0.705	0.407	0.270	0.328	0.449	0.363	0.636	0.611	1.000	0.663	0.379	0.269
NVDA	0.576	0.387	0.086	0.121	0.362	0.157	0.490	0.528	0.663	1.000	0.172	0.214
PG	0.365	0.233	0.571	0.545	0.322	0.630	0.404	0.192	0.379	0.172	1.000	0.225
XOM	0.311	0.560	0.317	0.296	0.553	0.377	0.448	0.186	0.269	0.214	0.225	1.000

This matrix results in following graph:

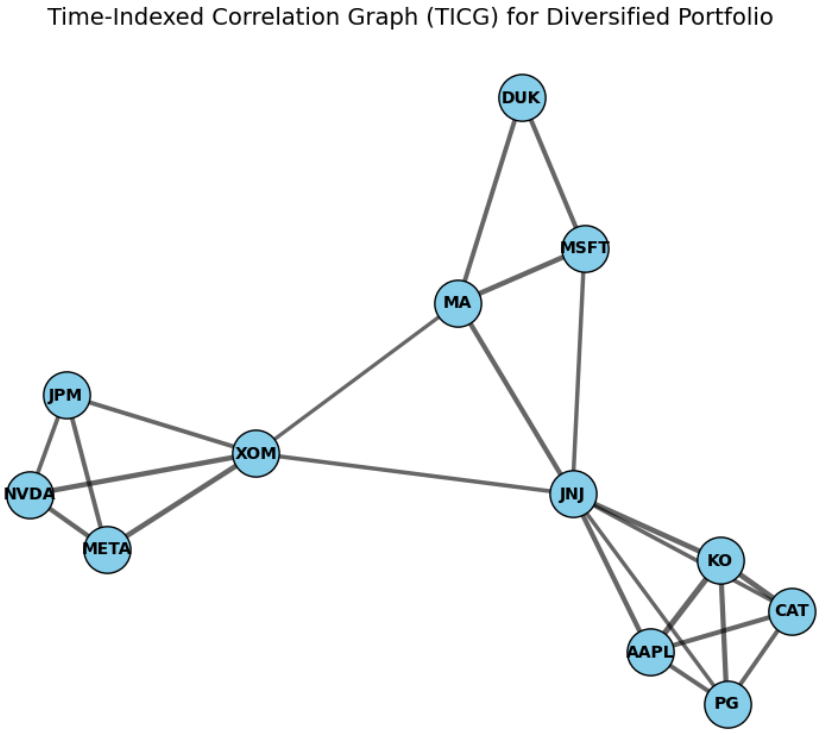


Figure 8: TIC Graph

9.3.2 QUBO Formulation

Now that the TICG is created, we need a QUBO Form to encode the graph. For the purpose of creating a quadratic equation, I declared the coefficient $A = 4$ as a reward for including a node in a clique, and $B = 2$ as a penalty for a pair of nodes that are not connected. Using these coefficients, we obtain the following QUBO matrix:

Table 5: QUBO Matrix Q for Maximum Clique Problem (A = 1, B = 2)

	AAPL	**JNJ**	**PG**	**KO**	**CAT**	**MSFT**	**MA**	**DUK**	**NVDA**	**JPM**	**XOM**	**META**
AAPL	-4.0	0.0	0.0	0.0	0.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0
JNJ	0.0	-4.0	0.0	0.0	0.0	0.0	0.0	2.0	2.0	2.0	0.0	2.0
PG	0.0	0.0	-4.0	0.0	0.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0
KO	0.0	0.0	0.0	-4.0	0.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0
CAT	0.0	0.0	0.0	0.0	-4.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0
MSFT	2.0	0.0	2.0	2.0	2.0	-4.0	0.0	0.0	2.0	2.0	2.0	2.0
MA	2.0	0.0	2.0	2.0	2.0	0.0	-4.0	0.0	2.0	2.0	0.0	2.0
DUK	2.0	2.0	2.0	2.0	2.0	0.0	0.0	-4.0	2.0	2.0	2.0	2.0
NVDA	2.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0	-4.0	0.0	0.0	0.0
JPM	2.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0	0.0	-4.0	0.0	0.0
XOM	2.0	0.0	2.0	2.0	2.0	2.0	0.0	2.0	0.0	0.0	-4.0	0.0
META	2.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0	0.0	0.0	0.0	-4.0

A visual reference of the matrix is as follows:

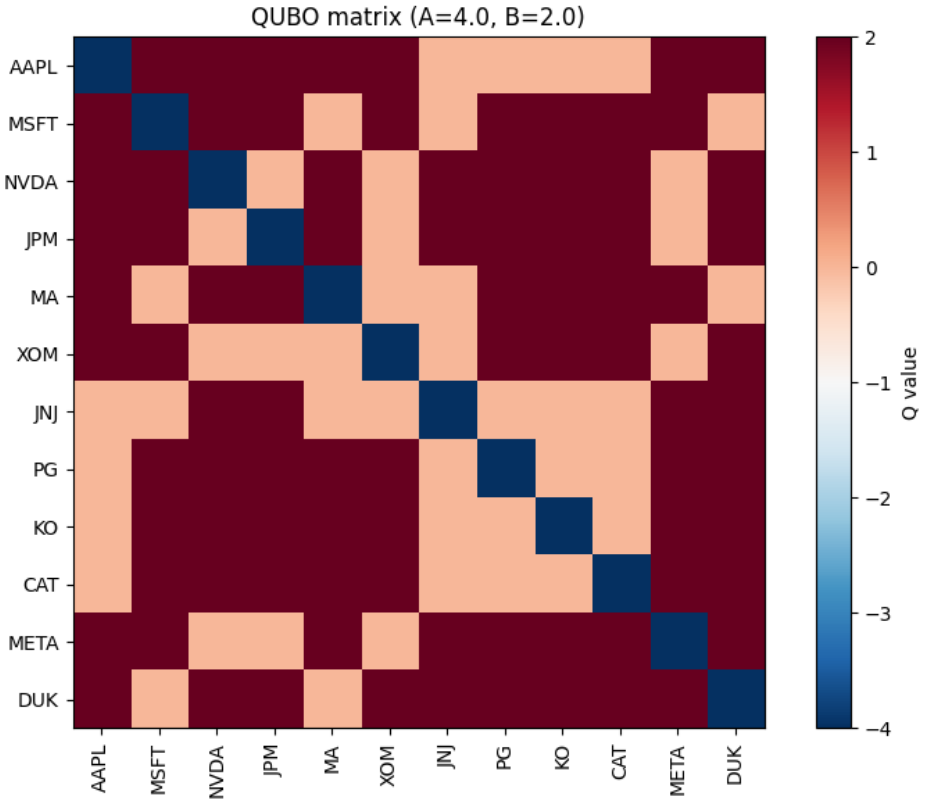


Figure 9: Visual representaion of correlation between assets

9.3.3 QAOA approach to solving QUBO

As mentioned before, during this reproduction, I attempt to obtain the ground state of the QUBO matrix. First, using the simple formula of $x_i = (Z_i - 1)/2$, we convert QUBO to an Ising function. Finally, using the statevector simulator of PennyLane along with AdamOptimizer as a gradient-based optimizer, we attempt to narrow down the solution by obtaining the solution for QUBO. In an attempt to increase accuracy, I set p = 5, depth, and number of iterations to 300, with a step size of 0.001. The result obtained after this is,

```
Starting QAOA optimization...
Iter 000 | Energy = 9.305564
Iter 010 | Energy = 6.877155
Iter 020 | Energy = 4.585518
...
Iter 290 | Energy = -9.060144
Iter 299 | Energy = -9.154519
Optimization finished.
Optimal params (gamma[0..], beta[0..]): [-0.08831498 -0.11460068  0.1267809   0.34583986  0.25468705
  0.16070094  0.03873923 -0.17583246 -0.15439607]
```

```
Sampling from optimized circuit (shots = Shots(total=2000))...

Most probable bitstrings and selected nodes (top 5):
100000111100 -> 142 samples (7.10%) | Selected: ['AAPL', 'JNJ', 'PG', 'KO', 'CAT']
001101000010 -> 119 samples (5.95%) | Selected: ['NVDA', 'JPM', 'XOM', 'META']
010010100001 -> 38 samples (1.90%) | Selected: ['MSFT', 'MA', 'JNJ', 'DUK']
100000011100 -> 36 samples (1.80%) | Selected: ['AAPL', 'PG', 'KO', 'CAT']
010011100001 -> 27 samples (1.35%) | Selected: ['MSFT', 'MA', 'XOM', 'JNJ', 'DUK']
```

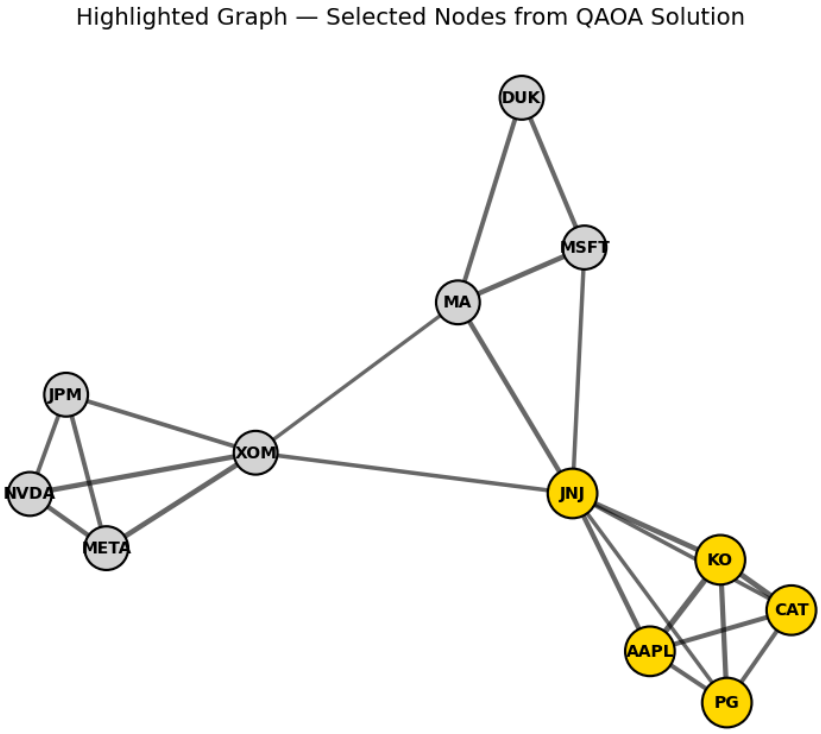


Figure 10: Graph with solution nodes highlighted

The solutions obtained through this method match the expected results for ['AAPL', 'JNJ', 'PG', 'KO', 'CAT']. Therefore, the accuracy of QAOA is established. However, as I mentioned at the start of this reproduction work, the reason for choosing only 12 assets was that it was much more manageable on computer hardware, specifically due to RAM limitations. If I attempt to increase the number of assets to 25, the RAM consumption exceeds 12 GB, which is still possible on more powerful hardware. However, once you approach 50 nodes, even the best hardware will struggle to process them properly.

This issue of memory limitation is primarily limited to the classical simulation of a quantum computer; however, the scalability issue remains in a quantum computer once noise and decoherence, qubit limitations, and increased runtime are considered. To address these issues, I attempted the following solutions.

9.3.4 Improving Scalability & Graph Clustering

To scale the algorithm for larger graphs, I utilized Graph Clustering, specifically Louvain Clustering, to create subgraphs or "communities" within the larger graph, thereby making it much more manageable. Using this, I was able to run a 25-node and even 50 50-node graph algorithm.

Louvain clustering (also called the Louvain method for community detection) is a greedy optimization algorithm that partitions a graph into communities (or clusters) such that nodes within the same community are densely connected, and nodes in different communities are sparsely connected.

It optimizes a measure called modularity, which quantifies the quality of a particular division of the network.

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \tag{13}$$

Where:

- A_{ij} = weight of the edge between nodes i and j (1 if an edge exists, 0 otherwise for unweighted graphs)
- k_i = degree (or total edge weight) of node i, given by $k_i = \sum_j A_{ij}$
- m = total number of edges in the network, given by $m = \frac{1}{2} \sum_{i,j} A_{ij}$
- c_i = community assignment of node i
- $\delta(c_i, c_j)$ = Kronecker delta, equal to 1 if $c_i = c_j$ (i.e., i and j belong to the same community), and 0 otherwise

A higher value of Q indicates a stronger community structure, meaning more edges fall within communities than would be expected by chance.

Using Louvain clustering on a large graph of 25 nodes I got following graph where differently colored nodes belong to different communities:

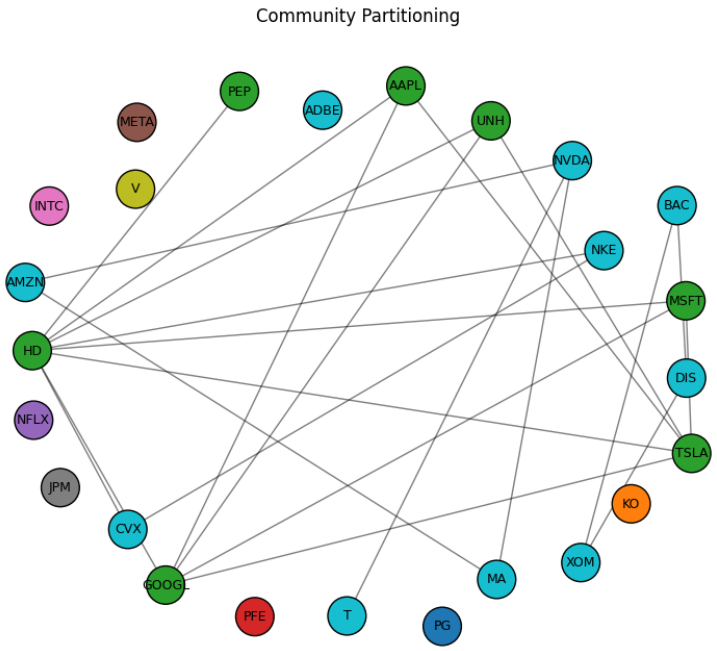


Figure 11: 25 node partitioned graph

Afterwards, I applied QAOA on each community separately. I considered the largest list of solutions among all the lists to be the final solution as seen in the graph below:

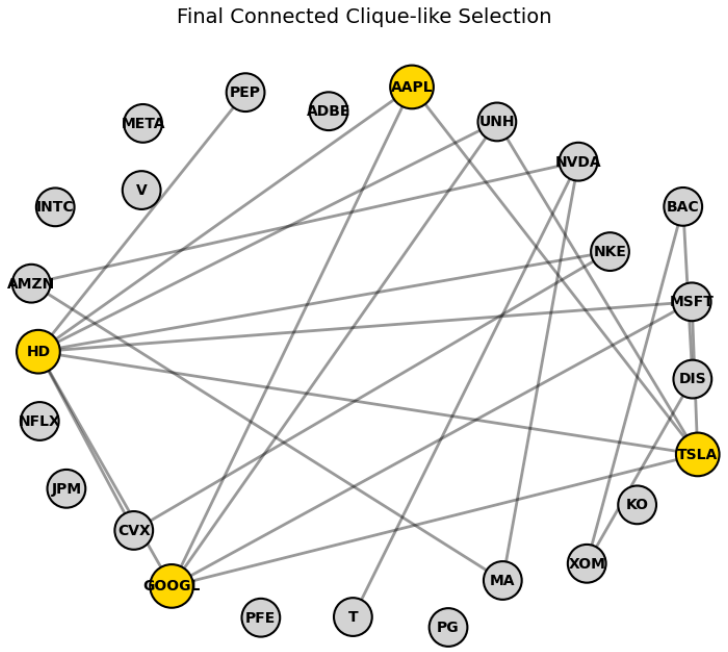


Figure 12: 25 node optimal solution highlighted

Similarly for 50 node graph:

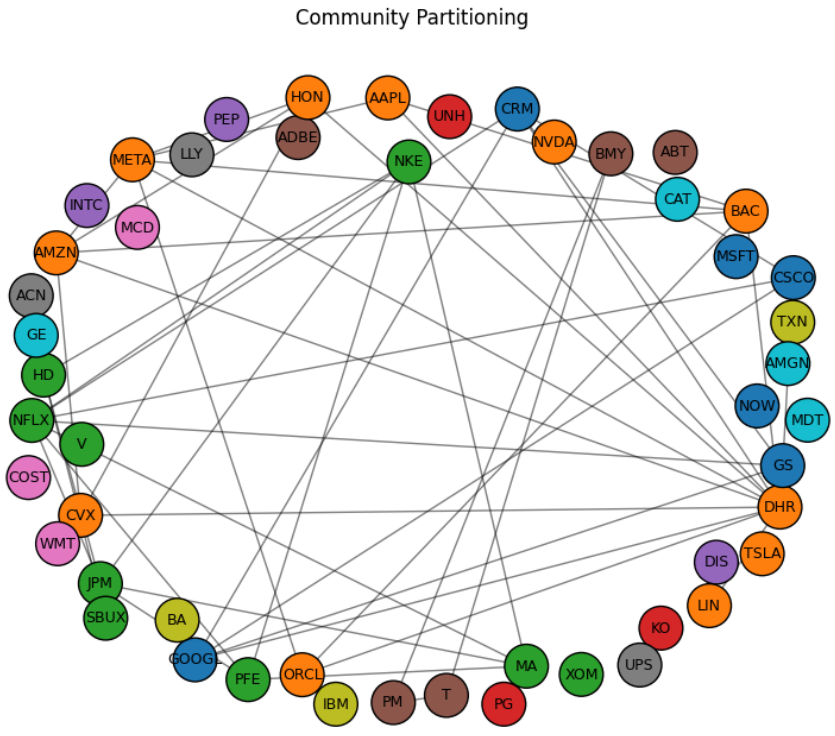


Figure 13: 50 node partitioned graph

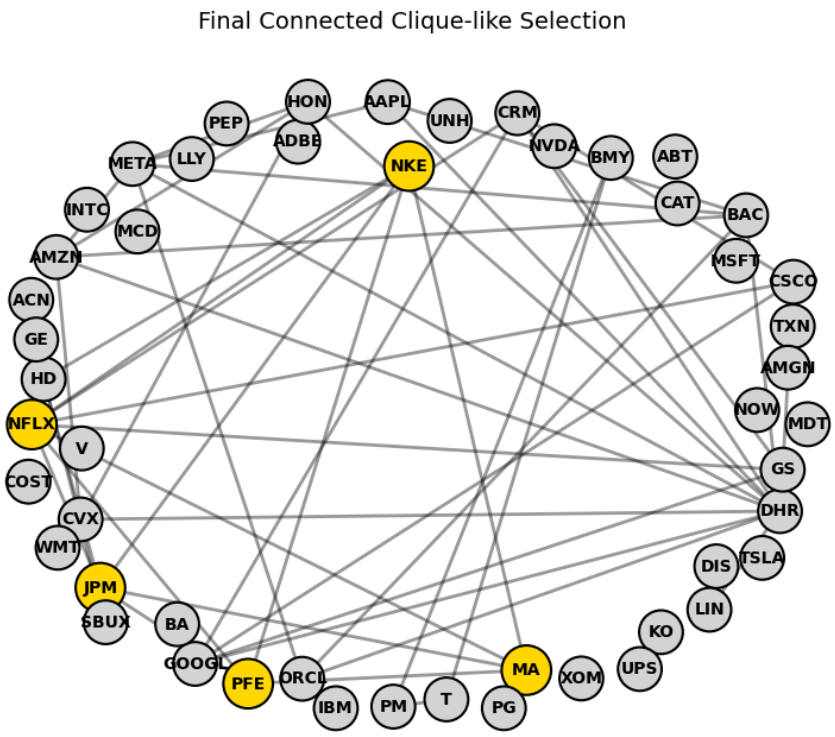


Figure 14: 50 node optimal solution highlighted

9.4 Result and Future Study

An attempt to replicate the Quantum Annealer result using QAOA was successful. I was also successfully able to solve the scalability issue on a smaller scale using the Graph clustering method. However, there are still significant issues that need to be studied in the future, i.e., the clustering method divides the graph into smaller communities, but if the maximum clique size is actually bigger than the largest community size, then scaling the solution obtained through such a method will only be a local maximum and not a global solution. There are methods, such as Greedy Search, that can check if connecting nodes can form a clique, which works well for smaller graphs. However, when considering graphs with 1000+ nodes, this method becomes computationally infeasible. This is the problem I would like to look into as part of future studies.

Here is a GitHub link to the code used for solving MCP using QAOA, I will keep on updating as I manage to improve the code: Git Link

10 Revision Notes

Revised on 12th Nivember 2025:

1. Replaced Figure 7(d) with correct graph.
2. Modified Section 9 to reflect progress in improving the QAOA algorithm with possibility of scaling beyond the hardware limitation. Added sub-subsection 9.3.4 along with more 25 and 50 node graphs.
3. Updated Github code with scalable algorithm